

BloodLink 2.0 — Implementation Plan (v2)

Prepared by: Zeeshan Sajid | zeeshansajid361@gmail.com

Team: Zeeshan Sajid & Irtaza | Stack: MongoDB, Express, React, Node (MERN) | Timeline: 4 weeks

0. Landscape Review — What Real Platforms Actually Do

Before revising the plan, it's worth knowing who else is solving this problem, because it changes what “hospital integration” should realistically mean.

- **Blood Link Pakistan** (bloodlink.pk) — an initiative of the Family and Fellows Foundation, registered with SECP and the Punjab Charity Commission. 900+ donors, 40+ facilitated donations, partners including NUST's Community Service Club, Air University, and Sundas Foundation Islamabad. Every request is manually reviewed by foundation staff against uploaded ID and a hospital-issued document before any donor is notified. Donors are gamified with verified-donation tiers.
- **E Blood** (eblood.com.pk) — AI/location-based matching; testimonials describe donors routed directly to named hospitals (e.g. Shifa International, Islamabad) for the actual donation.
- **Pakistan Red Crescent Society (PRCS)** — already runs a national Regional Blood Donor Program collaborating directly with hospitals and blood banks for donation camps, screening, and distribution.
- **Academic MERN precedents** (BloodConnect; a 2026 “Blood Bank Management System with AI Demand Forecasting” study) — validate MERN for this problem and contribute two ideas worth borrowing: QR-code donation check-in, and a simple moving-average forecast instead of a flat threshold.

Two takeaways that reshape this plan:

1. Even a registered, funded NGO doesn't attempt live hospital API integration — no one in this market does. Phase 4 below builds what actually works instead: hospitals as a verified location directory, manual document-based verification, and partnerships with organizations that already have hospital relationships.
2. The name “BloodLink” is already in active use by a registered Pakistani NGO with a Play Store app. Fine for a university project — but worth a one-line acknowledgment in your report, and don't publicly launch under this exact name without checking.

Phase Roadmap

Phase	Focus	Week
1	Foundation: architecture, design system, shared auth	Week 1 (first half)
2	Donor module (+ recognition levels)	Week 1 (2nd half) – Week 2
3	Seeker module (+ document-based verification)	Week 2
4	Hospital & Partner Network (practical model)	Week 3 (first half)
5	Admin & Verification Module	Week 3 (2nd half)
6	Real-time notifications, engagement, analytics	Week 4 (first half)
7	QR donation check-in (stretch)	Week 4 (parallel)

8	Integration, testing & deployment	Week 4 (2nd half)
---	-----------------------------------	-------------------

Hospital/partner outreach (Phase 4's real-world component) is emails and conversations, not code — start it in **Week 1**, in parallel with development.

Phase 1 — Foundation: Architecture, Design System & Core Authentication

Objective: Shared schema and a single auth system all four roles plug into.

Tools: Figma, Mermaid/Draw.io (ERD), MongoDB Atlas, Node.js, Express.js, Mongoose, JWT, bcryptjs, Helmet, express-rate-limit, dotenv, Nodemailer (free SMTP) for email verification.

Design system: a Figma “Design System” page — emergency red for alerts, a trusted medical blue for primary actions, consistent typography and button/badge states. Auto Layout so components map cleanly to Tailwind later.

Data model: one Users collection with a role enum (donor / seeker / hospital / admin) plus role-specific optional fields. Organizations, Inventory, and Requests stay separate.

Auth endpoints: POST /api/auth/register, POST /api/auth/verify-email, POST /api/auth/login, plus requireAuth and requireRole([...]) middleware.

Deliverables: ERD, Figma design system + wireframes for all four dashboards, Express scaffold with security middleware, User schema, auth routes + RBAC middleware.

Phase 2 — Donor Module

Objective: Donor registration, profile, eligibility, availability, dashboard.

Registration fields: full name, age (≥ 18), gender (drives cooldown), blood group, phone, email, password, city, last donation date (nullable).

Eligibility engine (WHO-standard, gender-aware cooldown)

- Male: 90-day cooldown · Female: 120-day cooldown
- nextEligibleDate = lastDonationDate + cooldown; immediately eligible if null
- Compute on read rather than storing a stale boolean — no cron job required for the MVP

```
const COOLDOWN_DAYS = { male: 90, female: 120 };

function getEligibility(gender, lastDonationDate) {
  if (!lastDonationDate) return { eligible: true, nextEligibleDate: null };
  const cooldown = COOLDOWN_DAYS[gender] ?? 90;
  const nextEligibleDate = new Date(lastDonationDate);
  nextEligibleDate.setDate(nextEligibleDate.getDate() + cooldown);
  return { eligible: new Date() >= nextEligibleDate, nextEligibleDate };
}
```

Availability vs. eligibility — keep separate. Only donors who are both eligible and available surface in search or Code Red alerts.

Donor recognition levels (borrowed from the real Blood Link Pakistan model): count each confirmed donation (see Phase 7) and assign a tier — Spark (1+), Pulse (3+), Life Saver (7+), Guardian (12+), Anchor (20+). Free, cosmetic, noticeably increases perceived polish.

Deliverables: GET/PUT /api/donors/me, PATCH /api/donors/me/availability, eligibility utility function, donor dashboard UI with eligibility card and level badge.

Phase 3 — Seeker Module

Objective: Registration/login, compatible-donor search, and a request flow that's actually verifiable.

Registration fields: name, phone, email, password, city.

Blood-group compatibility matrix

```
const COMPATIBLE_DONORS = {
  'O-': ['O-'],
  'O+': ['O+', 'O-'],
  'A-': ['A-', 'O-'],
  'A+': ['A+', 'A-', 'O+', 'O-'],
  'B-': ['B-', 'O-'],
  'B+': ['B+', 'B-', 'O+', 'O-'],
  'AB-': ['AB-', 'A-', 'B-', 'O-'],
  'AB+': ['A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-'], // universal recipient
};
```

Upgrade from v1: verifiable requests

The seeker selects a verified hospital, uploads a photo of the hospital-issued blood request slip (and optionally a CNIC), and selects the needed blood group. This replaces the unverifiable “Patient ID” text field with something an admin can actually check — the same trade-off real platforms make.

Free image storage: Cloudinary free tier (25 GB) — a `documentUrl` field on the Requests schema, no binary blobs in MongoDB.

Anonymous routing: donor sees only the hospital's contact/address; seeker sees only request status, never the donor's identity.

Deliverables: GET `/api/search`, POST `/api/requests` (with document upload), GET `/api/requests/mine`, compatibility matrix utility, seeker dashboard.

Phase 4 — Hospital & Partner Network Module (Practical Model)

Objective: Get real hospitals and organizations actually involved — realistically, not aspirationally.

No hospital's IT department will give a student project API access in a month — and per the landscape review, no comparable platform in this market has that either. Hospitals are modeled as a **verified location directory**, with an optional manual dashboard for staff who want to update stock themselves.

Two parallel tracks — both start in Week 1

- **Track A — Direct hospital contact** (lower priority, harder sell): reach out to hospitals already in your demo data (PIMS Islamabad, Shifa International, Rawalpindi hospitals). Ask for something small: permission to list them as a verified location, and staff willingness to use a simple manual dashboard. Don't lead with "API integration."
- **Track B — Organizational partnership** (higher priority, realistic in a month): contact the **PRCS regional blood donor program** (Islamabad/Rawalpindi chapter) — already hospital-connected nationally. Approach your **own university's community-service society** as your first verified partner organization, mirroring how NUST's and Air University's clubs partnered with the real Blood Link Pakistan.

Document every outreach attempt — who, what they said, the outcome. Even a polite decline is legitimate material for a "Real-World Validation" section in your report.

What still gets built regardless of outreach outcome

- Hospital registration → status: pending until admin approval
- Manual inventory dashboard (add/edit/delete: blood group, units, expiry) — the default and primary path
- Hashed API key + POST /api/inventory/sync, demoed via a mock hospital simulation script and labeled explicitly as a forward-looking capability, not a live hospital integration
- Code Red broadcast with a MongoDB TTL index (auto-expires ~6 hours), gated behind a confirmation modal

Deliverables: inventory + sync + broadcast routes, hospital dashboard UI, an outreach log (who/what/outcome) for your report.

Phase 5 — Admin & Verification Module

Objective: Platform oversight, playing the same "foundation staff" verification role the real platforms use.

- **Hospital verification queue:** approve (activates account + issues API key) or reject.
- **Request verification queue:** review each seeker's uploaded document before approving a request.
- **API key management:** view issued keys (masked), revoke instantly.
- **User/donor management:** search, list, block/unblock.
- **Analytics dashboard:** totals, units by blood group, low-stock list, most-requested group.

Deliverables: hospital + request approval routes, key revocation, user management, GET /api/admin/analytics, admin dashboard UI (Recharts or Chart.js).

Phase 6 — Real-Time Notifications, Engagement & Lightweight Analytics

- **Notification pipeline:** on an approved Request or Code Red, query donors matching bloodGroup/eligible/available/location, send a Web Push notification (self-generated VAPID keys — free) and write to an in-app fallback list.
- **Geocoding:** OpenStreetMap Nominatim (free) — cache city coordinates on first lookup.
- **Lightweight demand forecasting (stretch):** a 7-day moving average of unit consumption per blood group per hospital, flagged if trend suggests stock hits zero before the next expected cycle — a few lines of aggregation, not ML, but lets you honestly claim “predictive” alerts.

Deliverables: notification pipeline, Web Push setup, Nominatim caching layer, moving-average utility (stretch).

Phase 7 — QR Donation Check-In (Stretch)

Objective: Close a real gap from the original Java version — lastDonationDate was purely self-reported, so the eligibility engine could be gamed.

- **MVP version:** hospital/admin staff manually mark a request as “donation completed,” which updates lastDonationDate and recalculates eligibility and recognition level.
- **Stretch polish:** generate a QR code for the accepted request; staff scan it (html5-qrcode, free) to confirm in one tap — matches the real BloodConnect precedent.

Deliverables: PATCH /api/requests/:id/complete, QR generation (qrcode npm package) and scan-to-confirm UI (stretch).

Phase 8 — Integration, Testing & Deployment

- Postman collection covering every endpoint including failure paths (expired token, blocked user, invalid/revoked API key, unverified email, rejected document)
- Jest + Supertest unit tests for the eligibility engine and compatibility matrix
- Manual test matrix across all four dashboards mirroring your Sprint 3 Black-Box/White-Box structure
- Deploy backend → Render free tier, database → MongoDB Atlas free cluster, frontend → Vercel; update CORS/env vars, verify via Postman, then deploy frontend

Deliverables: live URL, exported Postman collection, test report, deployment checklist.

Suggested Division of Labor (2 People)

- **Backend & partnerships owner:** Phase 1, Phase 4 (API work and outreach emails together), Phase 5, Phase 8 backend half.
- **Frontend & UX owner:** Phase 1 design system, Phase 2, Phase 3, Phase 6/7 frontend half.

What Changed From v1 and Why

v1	v2	Why
Hospital API sync as the primary integration	Manual dashboard + directory as primary; API sync as a backup	Matches how every other platform in this market operates
“Patient ID” text field, unverifiable	Uploaded hospital slip + CNIC, admin-reviewed	Verifiable, matches real platforms' trust model

No donation confirmation loop	Manual/QR check-in updates lastDonationDate	Closes a real gap from the original Java version
Flat low-stock threshold only	Threshold + optional moving-average trend	Cheap way to sound predictive without ML
No engagement mechanic	Donor recognition levels	Free, borrowed from a real platform
No competitive/market context	Landscape review section	Shows real research, not just internal iteration